1 Introduction

LLMs are increasingly deployed as high-level planners that generate execution plans invoking tools, querying databases, manipulating files, and coordinating distributed workers and heterogeneous resources (e.g., GPUs and edge nodes). These plans must execute deterministically and safely within a runtime substrate. However, treating LLM output as directly executable code leads to systemic issues: (i) *structural fragility* (malformed JSON, mismatched blocks, invalid variable access), (ii) *uncontrolled effects* (unauthorized tools, unsafe filesystem writes, policy violations), and (iii) *nondeterminism and irreproducibility* (external tool calls, time, and filesystem interactions).

Key Insight. Treat LLM output as programs in a *restricted, verifiable IR*. CASLang constrains expressiveness to enable static verification and semantic guarantees, while still supporting structured control flow, tool invocation, filesystem operations, distributed spawning, and safe parallelization through DAG compilation.

Contributions (Summary).

- A restricted IR with a formal small-step semantics using oracle/trace modeling for external interactions.
- Static verification via well-formedness and effect-policy checking, with provable safety properties.
- A deterministic replay guarantee under fixed external traces.
- A semantics-preserving compilation to parallel DAG execution under explicit independence conditions.

2 CASLang Overview

CASLang is a JSONL-encoded IR with a finite instruction set; each instruction is a JSON object containing an `op` field and structured arguments. Blocks (e.g., conditionals, loops) are explicitly delimited.

Example (schematic):

```
{"op":"tool.call","tool_name":"seek_by_text",
 "input_text":"person in white shirt","top_k":10,"as":"r"}
{"op":"flow.return","to":"final","value":"${r}"}
```

Instruction categories include pure operations (`flow.set`, `dict.set`), control flow (`flow.if`, `flow.loop_start`), external operations (`tool.call`, `sandbox.exec`), filesystem operations (`fs.read_file`, `fs.write_file`), and distributed execution (`dist.spawn`, `dist.wait`). CASLang is designed to be (i) LLM-friendly (simple structure), and (ii) verifier-friendly (finite ops, static schemas).

3 Formal Model (Overview)

3.1 Syntax

A program is a finite sequence of instructions:

$$P \triangleq I_1; I_2; \ldots; I_n.$$

Each instruction I is drawn from a finite instruction set, e.g.:

$$I ::= \mathsf{Set}(x, e) \mid \mathsf{If}(e, P, P) \mid \mathsf{Loop}(x, e, P) \mid \mathsf{Return}(e) \mid \mathsf{ToolCall}(f, a, x) \mid \mathsf{FsRead}(p, x) \mid \ldots$$

JSONL is a serialization; the formal object is the parsed AST.

3.2 State and Trace

Execution configurations are:

$$\langle P, pc, \Sigma, T \rangle$$

where $pc \in \mathbb{N}$ is a program counter, $\Sigma = (Env, Ctrl, Out)$ is internal state, and T is an external trace. The environment maps variables to values:

$$Env : Var \to Value.$$

External nondeterminism is modeled by a finite trace:

$$T ::= e_1 \cdot e_2 \cdot \ldots$$

with events:

$$e ::= \mathsf{ToolResp}(f, a, r) \mid \mathsf{FsResp}(p, c) \mid \mathsf{TimeResp}(t).$$

3.3 Small-step Semantics

We define a transition relation:

$$\langle P, pc, \Sigma, T\rangle \longrightarrow \langle P, pc', \Sigma', T'\rangle.$$

Assignment (illustrative):

$$\frac{P[pc] = \mathsf{Set}(x, e)}{\langle P, pc, \Sigma, T\rangle \longrightarrow \langle P, pc + 1, \Sigma[x \mapsto [\![e]\!]_\Sigma], T\rangle}.$$

External tool call consumes one trace event:

$$\frac{T = \mathsf{ToolResp}(f, a, r) \cdot T'}{\langle P, pc, \Sigma, T\rangle \longrightarrow \langle P, pc + 1, \Sigma[x \mapsto r], T'\rangle}.$$

This oracle/trace modeling isolates external nondeterminism and enables replay reasoning.

4 Static Verification and Guarantees

4.1 Well-formedness

Let $WF(P)$ denote that P is structurally well-formed: schemas are valid, blocks are matched, variable references are syntactically valid, and block delimiters are properly paired.

Theorem 1 (Structural Soundness)**.** *If $WF(P)$ holds, then execution of P cannot reach a structural failure state. Formally, for any Σ_0 and T, if*

$$\langle P, 0, \Sigma_0, T\rangle \longrightarrow^* \langle P, pc, \Sigma, T'\rangle,$$

then either execution terminates normally or the next step is defined by the operational semantics.

4.2 Effects and Policy

Each instruction has an effect category:

$$\epsilon \ ::= \ Pure \mid IORead \mid IOWrite \mid External \mid Sandbox.$$

Let $Eff(P) = \bigcup_i eff(I_i)$ be the set of effects in P. Let Π be an allowed effect set (policy).

Theorem 2 (Policy Soundness)**.** *If $WF(P)$ and $Eff(P) \subseteq \Pi$, then execution of P produces no effect outside Π.*

4.3 Deterministic Replay

Theorem 3 (Replay Determinism)**.** *For any well-formed P and initial state Σ_0, and for any fixed trace T, execution is deterministic. If*

$$\langle P, \Sigma_0, T\rangle \longrightarrow^* (\Sigma_1, T_1) \quad \textit{and} \quad \langle P, \Sigma_0, T\rangle \longrightarrow^* (\Sigma_2, T_2),$$

then $\Sigma_1 = \Sigma_2$ and $T_1 = T_2$.

5 Parallel DAG Compilation (Overview)

CASLang programs are compiled into DAGs via (i) data-dependency analysis over *Env* reads/writes and (ii) effect-conflict analysis (e.g., disallow reordering conflicting I/O writes). Independent instructions may be reordered/parallelized.

Theorem 4 (Sequential Equivalence of DAG Execution)**.** *Let $DAG(P)$ be the dependency graph derived from P and let $Exec_{DAG}$ execute nodes in a topological order that respects dependencies and effect-conflict constraints. If $DAG(P)$ is acyclic and only independent instructions are reordered, then:*

$$Exec_{DAG}(P, \Sigma_0, T) \ = \ Exec_{Seq}(P, \Sigma_0, T).$$

6 Implementation and Evaluation (Preview)

We implement CASLang atop the CantorAI runtime, which provides distributed task scheduling, resource-aware placement, and fault-tolerant execution. CASLang supplies the verified control-plane IR, while CantorAI serves as the execution substrate.

Workloads. (i) tool-orchestration pipelines, (ii) distributed database retrieval, (iii) multi-node inference coordination.

Metrics. Structural failure rate (E1/E2), policy violations, end-to-end latency (p50/p95), throughput, and replay coverage.

Preliminary Results (placeholders). Across **N=XXX** LLM-generated scripts:

- Structural runtime failures reduced from **X%** to **0%**.
- Policy violations reduced from **X** to **0**.
- p95 latency improved by up to **Y%** via safe DAG parallelization.
- Deterministic replay coverage: **Z%** under recorded traces.

7 Conclusion

CASLang is a restricted, formally specified IR that enables static verification, policy safety, deterministic replay, and semantics-preserving parallel execution for LLM-driven distributed systems. Our implementation on CantorAI demonstrates that formal guarantees can coexist with practical distributed performance.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.